Agrasandhani: Smart Gateway Trade-offs for Stable IoT Dashboards under Last-Hop Impairment

Aman Jain
University of Illinois at Urbana-Champaign
Champaign, Illinois, USA
aman18@illinois.edu

Abstract

IoT dashboards are not only telemetry sinks; they are operator-facing views whose usefulness depends on how fresh, stable, and continuous the displayed state remains under impairment. This paper evaluates Agrasandhani, an MQTT-to-WebSocket smart gateway that reshapes sensor streams using fixed-window batching, latest-per-sensor compaction, and last-known-good state exposure. Using replicated Intel Berkeley Research Lab traces and a smaller Array of Things validation slice, the study compares V0 (naive forwarding), V2 (batching plus compaction), and V4 (smart path plus last-known-good semantics) under clean, bandwidth-capped, lossy, delay/jitter, and outage scenarios. In the Intel primary matrix, over the full measured `outage_5s` condition, V2 and V4 reduce downstream frames from 116 in V0 to 4 and 6, respectively, but increase clean-path p95 latency from 31.9 ms to 276.2 ms and 607.7 ms. V4 preserves visible state during outage, but at substantially higher age. The smart variants do not reduce downstream payload bytes in this setup, and the tested adaptive controller does not materially outperform fixed-window batching. The resulting claim is bounded: gateway-side pacing improves dashboard continuity and rendering stability when continuity is valued over minimum latency, but the cost is older visible state and no blanket byte-volume benefit.

Code Repository: https://github.com/AJMSD/Agrasandhani

1 Introduction

IoT dashboards are operational interfaces, not passive endpoints for telemetry. When every sensor update is forwarded immediately, the browser-facing path inherits bursty arrivals, repeated redraws, and abrupt visibility gaps during connectivity faults. For an operator monitoring a live deployment, the relevant failure mode is not only loss of transport efficiency, but loss of interpretability at the display. This distinction is especially important when impairment occurs on the final gateway-to-dashboard hop: the broker can remain healthy while the rendered view becomes erratic.

This paper studies that dashboard-facing problem in the measured path `dataset replay` → `MQTT broker` → `smart gateway` → `impairment proxy` → `WebSocket dashboard`. The central question is whether a gateway can trade immediacy for operator-visible stability in a controlled and measurable way. Here, stability means a paced downstream rendering stream with predictable behavior during outage and recovery, while freshness means the age of the visible state when the dashboard renders it.

Agrasandhani addresses this question with a gateway that batches sensor updates, compacts each batch to the latest reading per sensor, optionally adapts its batch window, and exposes last-known-good state with explicit age semantics. The empirical result is narrower than a generic optimization claim. The strongest measured effect is a large reduction in downstream frame cadence under last-hop impairment, especially during outage and recovery. That reduction does not imply lower payload-byte volume, and it comes with higher display latency and older visible state.

The paper makes three bounded contributions:

- It quantifies the primary gateway trade-off under last-hop impairment: smart variants sharply reduce downstream frame cadence and produce a steadier dashboard stream, but increase end-to-end latency.
- It distinguishes continuity from freshness by showing that last-known-good state improves visible outage behavior while increasing the age of displayed data.
- It reports negative and mixed results explicitly: downstream payload bytes do not decrease in this setup, the adaptive controller does not materially outperform fixed-window batching, and the measured benefit beyond batching alone is limited.

Together, these results evaluate when gateway-side pacing and retained-state semantics improve dashboard stability under last-hop impairment, and what they cost in latency, freshness, and payload volume.

2 Related Work

Work on edge and fog data reduction treats filtering, aggregation, and adaptation as lightweight ways to reduce communication pressure without relocating the full analytics stack into the network. Surveys by Pioli et al. and Sadri et al. emphasize both the practical appeal of these mechanisms and the need to report trade-offs rather than assume universal gains [8, 10]. Agrasandhani follows this line of work at the gateway layer, but evaluates policies through dashboard-facing outcomes: downstream frame cadence, display latency, and visible age under last-hop impairment.

MQTT-specific work motivates the treatment of delivery semantics, duplicates, and freshness. The MQTT 3.1.1 specification distinguishes QoS 0 and QoS 1 delivery guarantees, and Kovalchuk et al. note that at-least-once delivery can require application-side duplicate handling [2, 5]. Palmese et al. argue for adaptation under changing network conditions, while Yates et al. frame freshness as an explicit observer-side property rather than a by-product of throughput [7, 12]. These ideas align with Agrasandhani's batch-window control, compaction, and age-aware dashboard semantics, although the adaptive result measured here remains negative under the tested thresholds.

Methodologically, the paper is closer to controlled impairment studies than to open-ended deployment measurement. Jurgelionis et al. and Garcia and Hurtig argue that deterministic impairment

is useful because protocol and application behavior can be compared under repeatable degradations [3, 4]. Agrasandhani adopts this stance with a deterministic application-layer impairment proxy on the gateway-to-dashboard path, and treats proxy-side outputs as the source for dashboard-facing frame, byte, and stability measurements.

Broader pub/sub systems provide useful context but are not evaluated here. Kafka is designed as a distributed streaming platform with publish/subscribe, durable storage, and stream-processing capabilities, while RabbitMQ's AMQP model centers on exchanges, queues, bindings, and acknowledgements for routed messaging [1, 9]. Agrasandhani instead uses MQTT as the lightweight IoT ingest substrate and studies how a gateway transforms that stream before WebSocket visualization; Kafka- or RabbitMQ-backed deployments could still require a dashboard-facing shaping layer when the final browser hop is impaired.

3 System Design

3.1 End-to-End Pipeline

Agrasandhani is an MQTT-to-WebSocket pipeline that reshapes sensor updates before browser delivery. The measured system has five runtime components: a dataset replay publisher that emits normalized sensor messages, a Mosquitto broker that carries MQTT QoS 0 or QoS 1 traffic, a gateway that transforms the stream, an application-layer impairment proxy on the gateway-to-dashboard hop, and a WebSocket dashboard/browser that renders the processed stream. Replay messages carry `sensor_id`, `metric_type`, `msg_id`, and `ts_sent`; these fields support end-to-end latency measurement and downstream-policy comparison. Figure 1 shows the measured path.

This split is intentional. MQTT is used on the device-style ingest side between replayed sensors and the gateway, while WebSockets expose processed visualization state to the browser. The gateway is therefore the point where raw pub/sub telemetry is reshaped into dashboard-visible behavior.

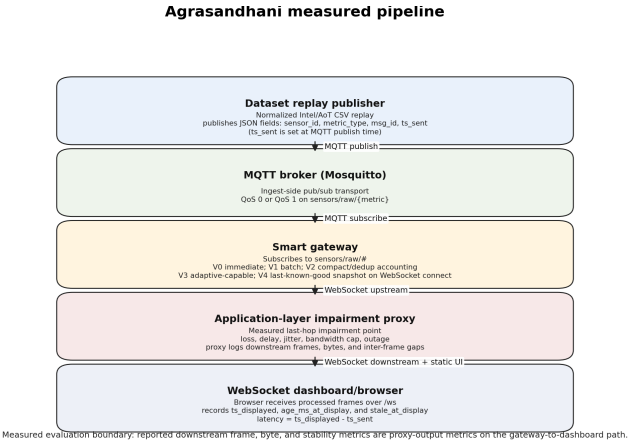

Figure 1: Agrasandhani measured end-to-end pipeline.

3.2 Gateway Policies

The gateway policy ladder spans V0 through V4. V0 forwards every message immediately and serves as the baseline. V1 introduces hybrid time/size batching. V2 keeps batching and compacts each batch to the latest update per sensor, while also exposing duplicate-sensitive accounting. V3 preserves V2's compaction logic and allows the effective batch window to change in response to queue depth and send-time signals. V4 runs on the same adaptive-capable smart path and adds last-known-good snapshot behavior with explicit freshness semantics and TTL-based staleness marking. In the measured runs, however, the adaptive counters stayed inert; observed V4 differences are therefore interpreted as retained-state effects on top of fixed-window pacing rather than as adaptive-control gains.

The main comparison focuses on V0, V2, and V4. V2 captures the primary pacing mechanism without last-known-good retention. V4 exposes the continuity behavior created by retaining visible state through outage. V1 and V3 remain supporting ablations because they isolate the incremental effects of batching and adaptation beyond the main comparison.

V4 should not be read as a faster or lower-traffic V2. V2 provides most of the measured pacing and cadence reduction; V4 adds retained-state continuity and the expected cost is older displayed data.

The main-matrix smart variants use a 250 ms base batch window and a 1000 ms freshness TTL. For adaptive-capable modes, the controller may vary the effective batch window between 10 ms and 1000 ms, increasing it by 100 ms when queue depth or send time crosses threshold and decreasing it by 50 ms after sustained healthy flushes. In the evaluated impairments, the supporting adaptive analysis shows that the controller did not move off the 250 ms operating point.

3.3 Measurement Boundary

In the reported runs, impairments are injected on the gateway-to-dashboard last hop through the application-layer impairment proxy. Optional host-level shaping exists in the repository, but the reported results are grounded in the proxy path. This boundary matters because downstream frame and byte metrics are measured at the proxy output. Dashboard logs provide display-time latency and age, proxy logs provide downstream frame, byte, and cadence metrics, and gateway metrics provide adaptive-window and duplicate counters. The central claims concern what the dashboard-facing path emits and displays, not generic broker-link throughput. MQTT remains relevant for ingest semantics and QoS handling, especially for duplicate-sensitive behavior, but the research object is the gateway's effect on the visualization stream seen by the operator.

4 Experimental Setup and Metrics

4.1 Workloads and Trial Matrix

The primary evidence source is the replicated Intel sweep root final-intel-primary-replicated-20260408-135251. It uses a bounded slice of the Intel Berkeley Research Lab deployment data [6], normalized into Agrasandhani's replay schema. The slice contains 4,320 replay rows drawn from 54 sensors. The primary matrix compares V0, V2, and V4 under MQTT QoS 0 and QoS 1 across five scenarios: clean, bandwidth_200kbps, loss_2pct, delay_50ms_jitter20ms, and outage_5s. Each Intel condition is replicated three times with shared impairment seeds (53701, 53702, and 53703) so variants experience the same randomized impairments.

The portability check is the replicated AoT sweep root final-aot-validation-replicated-20260408-135251, derived from a bounded Array of Things slice [11]. AoT is used because it represents a different urban environmental sensing source, while Intel remains the primary evidence source for final claims. That validation slice contains 520 normalized rows drawn from 36 sensors. The AoT matrix is intentionally smaller: it compares V0 and V4 under QoS 0 for the clean and outage scenarios with two trials per condition. The AoT results are treated as a directional consistency check rather than as a second primary claim source.

The replay harness accelerates both workloads by 5×. The replicated Intel and AoT runs also enable burst mode from replay second 5 through replay second 15 with an additional 8× burst multiplier. Intel runs use a 30 s wall-clock replay window and a 200-sensor target; AoT uses a 20 s wall-clock replay window and a 50-sensor target. Unless otherwise noted, in-body table values are condition means across replicated trials. Supporting ablations use the packaged replicated sweep roots for the V2 batch-window sweep, the V1 versus V2 isolation sweep, the V2 versus V3 adaptive comparison, and the V3 parameter sweep. In the QoS 0 headline matrix, the clean, bandwidth-capped, and outage frame-count patterns repeated exactly across trials, while the lossy case varied only at the one-frame level and preserved the large gap between V0 and the smart variants.

The supporting sweeps are narrower by design. The V2 batch-window study varies only the fixed batch window while keeping the clean Intel workload and QoS 0 constant. The V1 versus V2 isolation sweep compares batching alone against batching plus compaction under matched batch windows for the clean, bandwidth-capped, and outage scenarios. The adaptive studies reuse the same Intel workload to compare V2 and V3 directly, then expand to a parameter grid over the send-slow threshold, step-up increment, and maximum adaptive window. These sweeps interpret mechanism-level behavior; they do not replace the primary Intel matrix.

This matrix is designed to support a bounded dashboard-facing claim rather than a full deployment-general performance characterization.

4.2 Impairment Model and Metrics

Impairments are applied on the gateway-to-dashboard hop through the deterministic application-layer proxy. The clean scenario applies no impairment. The bandwidth-capped scenario limits the hop to 200 kbps. The loss scenario injects 2% packet loss. The delay/jitter scenario applies 50 ms delay with 20 ms jitter. The outage profile is phase-based, with 10 s of steady delivery, a 5 s complete outage, and a 45 s recovery interval. The repository also includes optional host-level shaping support, but the reported results use the proxy-side impairment path and its outputs.

Latency is measured as `ts_displayed - ts_sent` and summarized with mean, p50, p95, and p99. Downstream traffic is measured at the impairment proxy as emitted frames and payload bytes. Freshness is interpreted as dashboard age of information. The primary proxy-side stability metric is proxy frame-rate variability, with proxy inter-frame gap used as cadence context; dashboard row cadence is not treated as a frame clock because the browser records rendered events rather than display refreshes. For outage analysis, age over time is more informative than stale fraction alone because idle stale transitions are not sampled when the dashboard does not render new rows.

5 Results

5.1 Primary Intel Results

The strongest measured effect is downstream cadence reduction under last-hop impairment. Across the four headline Intel QoS 0 scenarios, V0 emits between 116 and 132 downstream frames, whereas V2 and V4 emit between 4 and 6 (Table 1). Figure 2 shows the same effect over the outage scenario. Over the full measured `outage_5s` condition, V0 emits 116 downstream frames; V2 emits 4 and V4 emits 6, corresponding to reductions of 96.6% and 94.8%. The gateway therefore reshapes a bursty dashboard stream into a paced sequence of aggregate updates.

Table 1: Primary Intel summary for QoS 0 across three replicated trials. V2 and V4 sharply reduce proxy-emitted downstream frames, but the same rows show higher display latency and no payload-byte reduction.

Variant	Frames	Bytes	Latency p95 (ms)	Scenario
V0	132	12069	31.9	clean
V2	5	13051	276.2	clean
V4	6	17641	607.7	clean
V0	132	12069	121.5	bandwidth_200kbps
V2	5	13051	271.5	bandwidth_200kbps
V4	6	17641	573.7	bandwidth_200kbps
V0	128	11725	115.7	loss_2pct
V2	4	12129	267.0	loss_2pct
V4	5	16845	546.0	loss_2pct
V0	116	10605	119.8	outage_5s
V2	4	11401	279.1	outage_5s
V4	6	17641	541.7	outage_5s

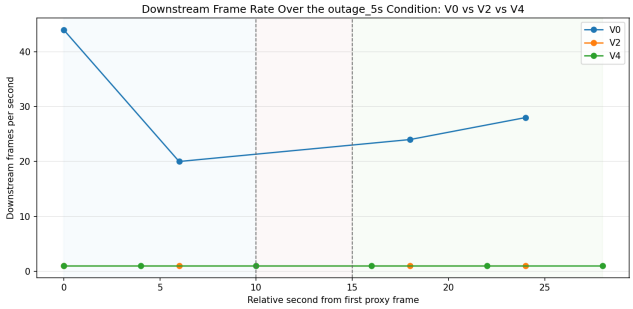

Figure 2: Proxy-emitted downstream frame rate during the Intel `outage_5s` QoS 0 condition. V2 and V4 replace V0's bursty outage/recovery stream with a much lower, steadier cadence.

That pacing comes at a latency cost. In the clean path, V0 reaches 31.9 ms p95 latency, compared with 276.2 ms for V2 and 607.7 ms for V4 (Figure 3 and Table 1). The positive result is therefore not faster delivery, but steadier rendering with older displayed data.

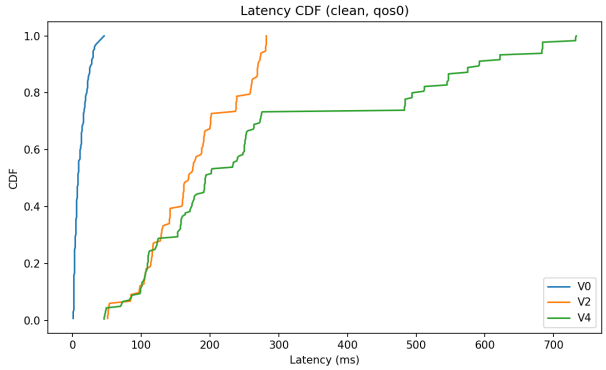

Figure 3: Clean-path browser-display latency CDF for Intel QoS 0. Pacing shifts V2 and V4 right relative to V0, showing that dashboard stability is purchased with higher display latency.

The bandwidth-capped case makes the mechanism more specific. Relative to clean, the 200 kbps cap raises V0 p95 latency by 89.6 ms, but leaves V2 and V4 within 35 ms of their clean-path p95 values because those variants already throttle downstream cadence before the impaired hop. The cap therefore hurts the raw stream more than the paced stream, even though the paced variants still send at least as many downstream payload bytes as V0. The delay/jitter condition is interpreted in text and Figure 4, rather than added to the condensed primary Intel table, because its main value is the shape of proxy timing variation.

Table 2: Proxy-side stability metrics for Intel delay/jitter QoS 0.

Variant	Gap p95 (ms)	Frame-rate stddev (s^{-1})	Max frame rate (s^{-1})
V0	79.8	3.81	18
V2	6071.0	0.00	1
V4	6068.3	0.00	1

The proxy-side stability metrics point in the same direction. Table 2 uses proxy frame-rate variability as the primary proxy-side stability metric and inter-frame-gap p95 as cadence context. Under Intel QoS 0 delay/jitter, V0 has a proxy frame-rate standard deviation of 3.81 s^{-1} and reaches 18 frames/s. V2 and V4 reduce the measured frame-rate standard deviation to 0.00 s^{-1} and never exceed 1 frame/s. At the reported precision, the smart variants appear as a near-constant low-rate stream. Their inter-frame-gap p95 rises to about 6.1 s because the gateway emits a much lower-rate sequence. The smart path therefore reduces visible burstiness by pacing the dashboard stream, not by removing timing variation from the impaired hop itself.

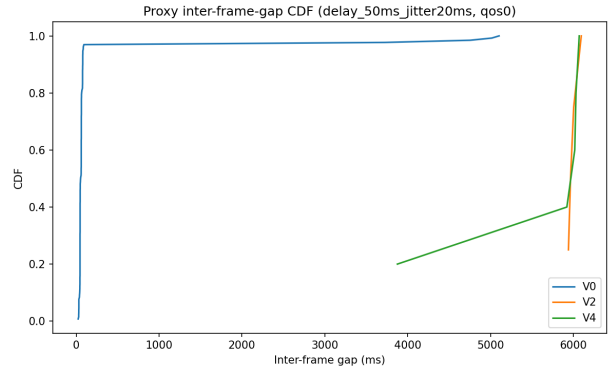

Figure 4: Proxy inter-frame-gap CDF for Intel QoS 0 under delay/jitter.

Delay and jitter expose a complementary effect. In delay_50ms _jitter20ms, V0 latency p95 rises to 1838.4 ms, whereas V2 and V4 remain at 324.7 ms and 645.4 ms while emitting only five and six downstream frames. The gateway converts a bursty raw stream into a low-rate downstream sequence whose visible latency grows far less sharply under per-packet timing disturbance.

V4 improves continuity rather than freshness. In the representative QoS 0 phase split in Table 3, V0 records no browser display samples whose display time falls in the outage window, while V4 records 20 retained-state samples crossing that window. The retained view is older: pre-outage age p95 rises from 45.0 ms in V0 to 489.0 ms in V4, and recovery age p95 rises from 125.15 ms to 278.3 ms. Figure 5 shows the per-frame mean age trace, while Table 3 reports p95 age values and display-sample counts for the representative trial. V2 delivers most of the cadence reduction without retained-state semantics; V4 is useful when a stable last-known-good view is worth the additional age.

Table 3: Representative Intel QoS 0 outage trace split by scenario phase.

Variant	Pre-outage age p95 (ms)	Outage display samples	Recovery age p95 (ms)	End stale rows/latest rows
V0	45.0	0	125.15	80/80
V4	489.0	20	278.3	100/100

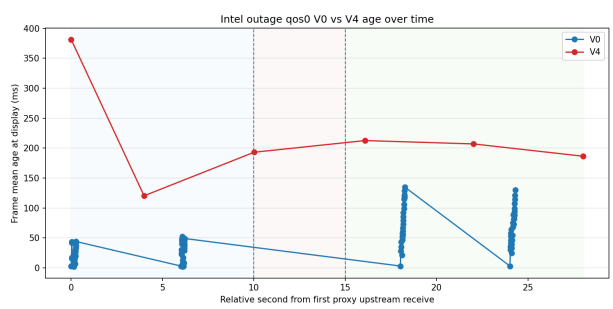

Figure 5: Outage age trace for Intel QoS 0, shown as mean browser-side age per rendered frame over time. V4 keeps retained state visible through outage, but the visible state is older than V0 before and after recovery.

Comparing V2 and V4 clarifies where the extra cost originates. V4 emits only one or two more frames than V2 in the headline

scenarios, so its additional benefit is not further cadence reduction. Instead, V4 layers retained-state continuity on top of batching. That continuity costs another 260–330 ms of p95 latency, depending on the scenario, and raises the age of the visible state both before recovery and after the hop becomes healthy again.

5.2 Supporting Results and Negative Findings

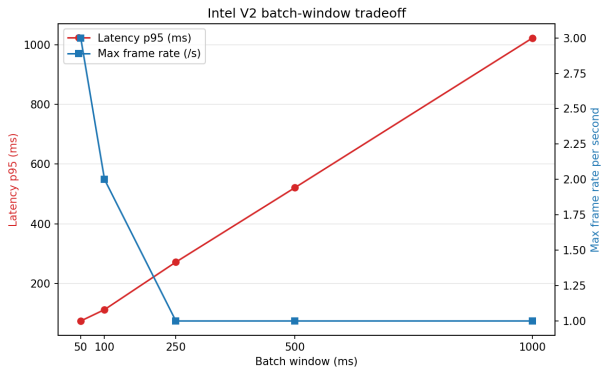

Figure 6: V2 batch-window trade-off; the 250 ms point captures most cadence reduction before latency rises sharply.

The batch-window sweep clarifies where the fixed-window operating point comes from. In the clean Intel V2 sweep, increasing the batch window from 50 ms to 1000 ms raises latency p95 from 74.5 ms to 1022.6 ms. The maximum downstream frame rate falls from 3 s^{-1} at 50 ms to 1 s^{-1} by the 250 ms operating point, then remains at 1 s^{-1} for the 500 ms and 1000 ms settings. The extreme 1000 ms setting also introduces a non-zero stale fraction. Larger windows therefore mainly add latency after 250 ms rather than producing meaningful further cadence reduction.

The byte audit rejects a blanket payload-reduction interpretation. In the four headline Intel QoS 0 scenarios, V2 increases downstream payload bytes by 3.4–8.1% relative to V0, and V4 increases them by 43.7–66.3%. Aggregation reduces the number of emitted WebSocket frames, but each aggregate frame carries a larger envelope and can include multiple compacted or retained sensor values. Frame reduction and payload-byte reduction are therefore separate outcomes. These results support reduced downstream frame cadence and burstiness, not reduced downstream payload-byte volume.

The isolation and adaptive sweeps narrow the mechanism-level story. In the V1 versus V2 isolation sweep, matched batch windows often yield identical downstream frames and bytes, and duplicate-sensitive counters remain at zero. Where differences appear, they are small and mixed: under clean conditions V2 changes downstream bytes relative to V1 by -13.0% to 0.0%, while under outage it ranges from 0.0% to +18.5%. These ablations identify fixed-window pacing, rather than compaction or adaptation alone, as the dominant mechanism behind the measured dashboard-facing improvement.

Likewise, the V2 versus V3 adaptive comparison shows no material advantage for the tested controller: stale fraction, frames, and bytes remain unchanged, the effective batch window stays fixed at 250 ms, and the broader parameter sweep yields no scenario in which the controller leaves that operating point and improves stability without violating the latency or byte guardrails.

As a supplemental combined-stress check, we also ran the Intel QoS 0 workload under `stress_combined`, which applies a 100 kbps last-hop cap, 5% loss, 100 ms delay, 40 ms jitter, and a 5 s outage in one scenario. The same frame-churn result persisted: V0 emitted 109.3 downstream frames on average, while V2 and V4 emitted 5.0 and 5.7. The harsher stacked impairment made naive forwarding queue badly, so p95 display age was 3407.7 ms for V0 versus 388.7 ms for V2 and 450.3 ms for V4; this should be read as a pacing result, not a general freshness win. Payload bytes still do not support a blanket byte-reduction claim: V0, V2, and V4 emitted 9,999, 11,938, and 15,734 downstream bytes. V4 ended with 92 retained latest rows marked stale, while the browser export recorded no rendered samples inside the 10–15 s outage window, so this check reinforces the bounded interpretation that Agrasandhani preserves pacing and retained state under stacked impairment, but not guaranteed freshness or uninterrupted rendering.

Table 4: Intel QoS sensitivity: QoS 1 minus QoS 0 ranges.

Variant	QoS1-QoS0 p95 range (ms)	Frame delta (%)	Byte delta (%)	Exact dup drops
V0	-8.1 to +68.5	0.0 to 0.0	0.0 to 0.0	0
V2	-6.9 to +4.0	0.0 to 0.0	0.0 to 0.0	0
V4	-56.7 to +55.7	-11.1 to +5.9	-7.6 to +0.3	0

Table 4 shows that QoS 0 and QoS 1 remain close in the primary matrix. QoS 1 produces zero observed exact duplicate drops, and it does not change the qualitative ranking of V0, V2, and V4 across the headline scenarios. For the dashboard-facing conclusion, the stability trade-off is driven by gateway pacing and retained-state behavior, not by MQTT delivery mode in this local broker setup.

Table 5: AoT portability check, not a primary claim matrix. The smaller QoS 0 slice shows the same V0-to-V4 cadence gap and latency cost on a second environmental sensing source.

Scenario	Variant	Frames	Latency p95 (ms)
clean	V0	439	77.5
clean	V4	18	276.1
outage_5s	V0	424	74.7
outage_5s	V4	16	273.2

Table 5 shows that the same continuity-versus-latency trade-off persists on the smaller AoT portability check, but the matrix remains too small to carry a second primary claim. The small V0 clean-to-outage frame delta is a timing consequence of the front-loaded AoT replay: each V0 outage proxy log contains 422 sent frames before the 10–15 s outage phase, 15 outage-phase drops, and 2 sent frames after recovery. The portability signal is the V0-to-V4 cadence gap and latency cost rather than a large clean-to-outage frame-count shift.

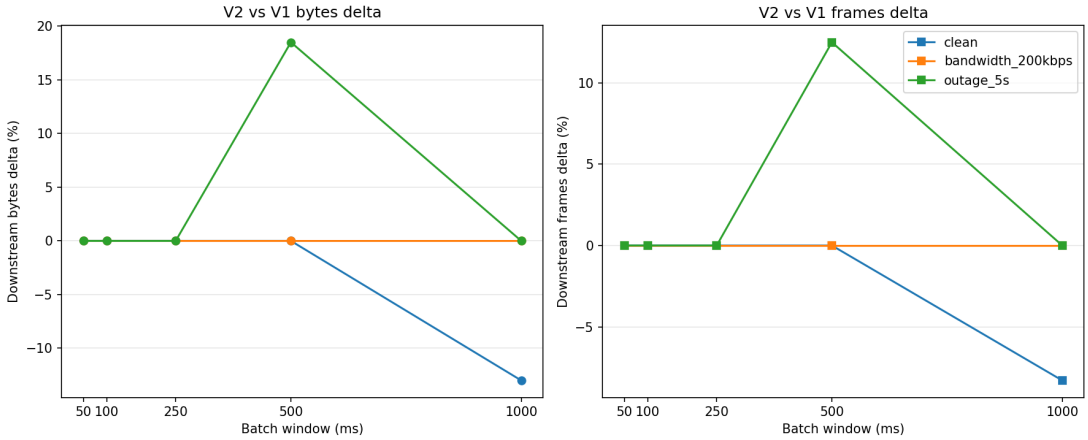

Figure 7: V1 versus V2 isolation sweep. Under matched batch windows, batching explains most of the observed effect.

6 Discussion and Threats to Validity

The final evidence supports a dashboard-facing systems interpretation. Agrasandhani is most useful when the design objective is dashboard continuity under last-hop impairment rather than minimum per-message latency. The dominant mechanism is pacing: the gateway emits far fewer downstream frames and makes the rendered stream easier to follow. V4 extends that behavior by preserving last-known-good state through outage. The trade-off is explicit throughout the results: display latency rises, visible age rises, and downstream payload bytes do not fall. The contribution is therefore not generic communication efficiency, but a controlled reshaping of operator-visible behavior under impairment.

6.1 Threats to Validity

Several limits bound the generality of the result. First, the main evidence comes from bounded Intel and AoT slices executed on a local, machine-scale testbed; the results should not be read as deployment-wide claims about larger workloads or different gateway placements. Second, the reported downstream metrics are proxy-side measurements on the gateway-to-dashboard hop. That boundary is appropriate for a dashboard-facing question, but it does not establish a generic broker or WAN result. For V4 specifically, the proxy is the gateway's WebSocket client in the measured path, so the gateway-side snapshot is exercised at proxy connection; browser reconnect-through-proxy semantics are not separately measured as direct browser reconnects to the gateway. Third, the QoS observations are setup-specific: the local broker configuration produces zero observed exact duplicate drops, so the QoS discussion remains descriptive rather than universal. Finally, missing-update attribution is often unavailable once aggregation changes proxy and gateway frame counts, which limits fine-grained causal claims about every omitted dashboard update. These limits do not invalidate the primary result, but they constrain it to a dashboard-facing, last-hop impairment study.

7 Conclusion

Agrasandhani shows that the value of a smart IoT gateway depends on which dashboard property the system prioritizes. In the replicated workloads studied here, gateway-side pacing made the browser-facing stream materially steadier and, in V4, preserved visible state through disruption. The cost was equally clear: higher display latency, older visible state, no blanket downstream byte reduction, and no material gain from the tested adaptive controller. For dashboard paths exposed to last-hop impairment, the design lesson is to treat the gateway as a display-shaping component rather than as a universal throughput optimizer.

References

[1] Apache Software Foundation. 2025. Apache Kafka Introduction. https://kafka.apache.org/24/getting-started/introduction/ Accessed: 2026-04-25.

[2] Andrew Banks and Rahul Gupta. 2014. MQTT Version 3.1.1. OASIS Standard. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html Accessed: 2026-02-18.

[3] Johan Garcia and Per Hurtig. 2016. KauNetEm: Deterministic Network Emulation in Linux. In *Proceedings of NetDev 1.1: The Technical Conference on Linux Networking*. Seville, Spain. https://www.diva-portal.org/smash/get/diva2:1069193/FULLTEXT01.pdf

[4] Audrius Jurgelionis, Jukka-Pekka Laulajainen, Matti Hirvonen, and Alf Inge Wang. 2011. An Empirical Study of NetEm Network Emulation Functionalities. In *Proceedings of the International Conference on Computer Communications and Networks (ICCCN)*. doi:10.1109/ICCCN.2011.6005933

[5] Oleksandr Kovalchuk, Yuri Gordienko, and Sergii Stirenko. 2018. Impact of MQTT Based Sensor Network Architecture on Delivery Delay Time. arXiv:1812.03296. https://arxiv.org/abs/1812.03296

[6] Samuel Madden. 2004. Intel Lab Data. https://db.csail.mit.edu/labdata/labdata.html 54-sensor Intel Berkeley Research Lab deployment page.

[7] Fabio Palmese, Alessandro E. C. Redondi, and Matteo Cesana. 2022. Adaptive Quality of Service Control for MQTT-SN. *Sensors* 22, 22 (2022), 8852. doi:10.3390/s22228852

[8] Laercio Pioli, Carina F. Dorneles, Douglas D. J. de Macedo, and Mario Antonio Ribeiro Dantas. 2022. An overview of data reduction solutions at the edge of IoT systems: a systematic mapping of the literature. *Computing* 104, 8 (2022), 1867–1889. doi:10.1007/s00607-022-01073-6

[9] RabbitMQ. 2026. AMQP 0-9-1 Model Explained. https://www.rabbitmq.com/tutorials/amqp-concepts Accessed: 2026-04-25.

[10] Ali Akbar Sadri, Amir Masoud Rahmani, Morteza Saberikamarposhti, and Mehdi Hosseinzadeh. 2022. Data reduction in fog computing and internet of things: A systematic literature survey. *Internet of Things* 20 (2022), 100629. doi:10.1016/j.iot.2022.100629

[11] Shaowen Wang, Fangzheng Lyu, Shaohua Wang, Charles Catlett, Anand Padmanabhan, and Kiumars Soltani. 2021. Data for Integrating CyberGIS and Urban Sensing for Reproducible Streaming Analytics. doi:10.13012/B2IDB-0286574_V1

[12] Roy D. Yates, Yin Sun, D. Richard Brown, Sanjit K. Kaul, Eytan Modiano, and Sennur Ulukus. 2021. Age of Information: An Introduction and Survey. *IEEE Journal on Selected Areas in Communications* 39, 5 (2021), 1183–1210. doi:10.1109/JSAC.2021.3065072